Delivery Buddy Backend

Requirements Specification

Backend API Internship Assessment

Prepared by: Chewachong Larry Che

GitHub: Larry-Craig | chewachongcraig@gmail.com

July 2026

1. Project Overview

Delivery Buddy Backend is a RESTful API built with NestJS and Prisma ORM that powers the Delivery Buddy mobile application. It manages driver registration and authentication, work shift tracking, delivery order lifecycle, an in-app support chat per order, and a digital wallet system covering balances, withdrawals, transaction history, and cached exchange rates.

The service follows a modular NestJS architecture, with each domain (Drivers, Orders, Shifts, Wallet) isolated into its own module, controller, and service. Data is persisted using Prisma ORM on top of SQLite (via the Better SQLite3 driver), and all endpoints are documented interactively through Swagger UI.

Base URL

```
http://localhost:3000/v1
```

Response Format

Successful requests return JSON representing the requested resource.

```
{
  "id": "f2d6a1c4-...",
  "name": "John Doe"
}
```

Errors follow a consistent NestJS validation-error shape:

```
{
  "statusCode": 400,
  "message": "Validation failed",
  "error": "Bad Request"
}
```

2. Architecture & Data Model

Persistent storage is implemented with Prisma ORM over a SQLite database file (prisma/dev.db), accessed through the Better SQLite3 driver for synchronous, low-latency reads and writes. The schema below models five core entities: User (driver), Shift, Order, Transaction, and ChatMessage.

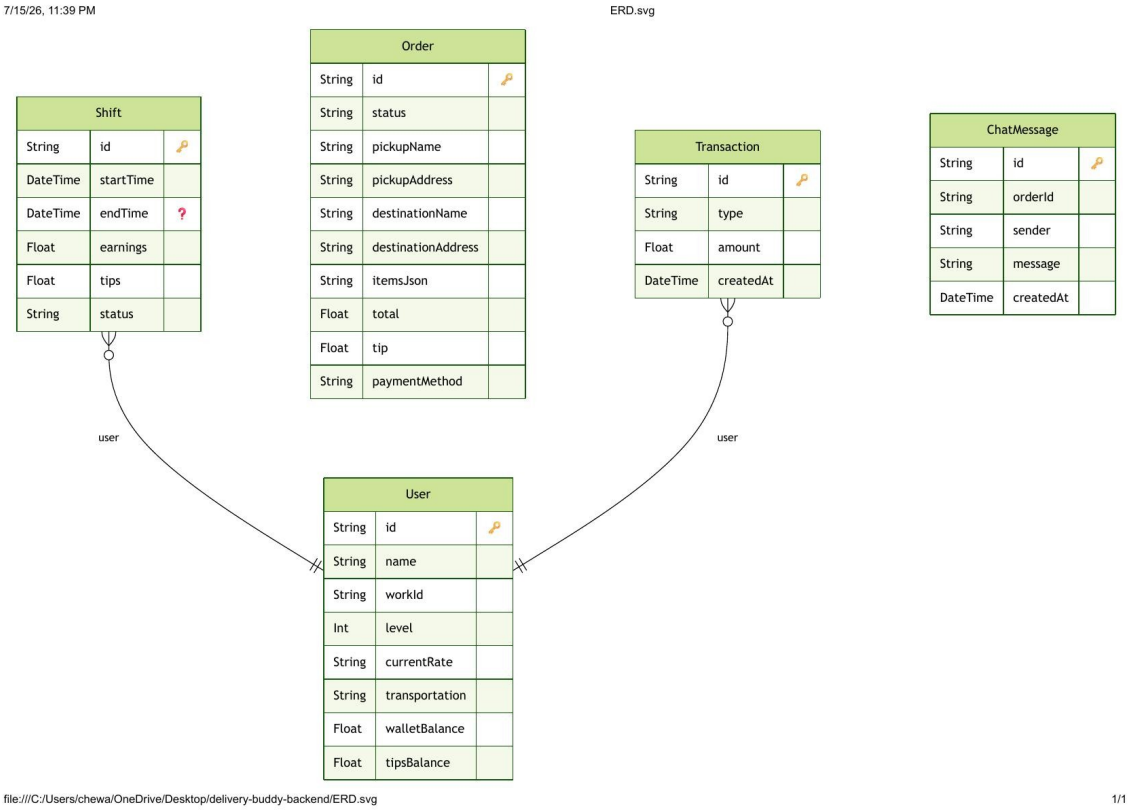

Figure 1 — Entity Relationship Diagram (Prisma schema)

Entity Summary

Entity	Description
User	Driver account: id, name, workId, level, currentRate, transportation, walletBalance, tipsBalance.
Shift	A driver's work session: id, startTime, endTime, earnings, tips, status.
Order	A delivery order: id, status, pickup/destination details, itemsJson, total, tip, paymentMethod.
Transaction	A wallet ledger entry: id, type, amount, createdAt.
ChatMessage	A support message tied to an order: id, orderId, sender, message, createdAt.

3. Drivers Module

Method	Endpoint	Purpose
POST	`/v1/drivers/register`	Register a new driver
GET	`/v1/drivers/login`	Log in a driver using their Work ID
GET	`/v1/drivers/:id`	Retrieve a driver's profile and wallet summary

Register Driver

Endpoint: POST /v1/drivers/register

Purpose: Registers a new delivery driver and creates their wallet.

Request Body

```
{
  "name": "John Doe",
  "workId": "DRV-123"
}
```

Response

```
{
  "id": "1",
  "name": "John Doe",
  "workId": "DRV-123",
  "createdAt": "2026-07-16T10:00:00.000Z"
}
```

Driver Login

Endpoint: GET /v1/drivers/login?workId=DRV-123

Response

```
{
  "id": "1",
  "name": "John Doe",
  "workId": "DRV-123"
}
```

Driver Profile

Endpoint: GET /v1/drivers/:id

Response

```
{
  "id": "1",
  "name": "John Doe",
  "workId": "DRV-123",
  "wallet": {
    "balance": 150.50
  }
}
```

4. Shifts Module

Method	Endpoint	Purpose
POST	/v1/shifts/start	Start a new work shift

POST	/v1/shifts/stop	End the current active shift
GET	/v1/shifts/active	Retrieve the driver's current active shift
GET	/v1/shifts/last	Retrieve the driver's previous (completed) shift

Start Shift

Endpoint: POST /v1/shifts/start

Request Body

```
{
  "driverId": "1"
}
```

Response

```
{
  "id": "1",
  "driverId": "1",
  "startTime": "2026-07-16T10:00:00.000Z",
  "endTime": null,
  "isActive": true
}
```

Stop Shift

Endpoint: POST /v1/shifts/stop

Request Body

```
{
  "driverId": "1"
}
```

Response

```
{
  "id": "1",
  "driverId": "1",
  "startTime": "2026-07-16T10:00:00.000Z",
  "endTime": "2026-07-16T18:00:00.000Z",
  "isActive": false
}
```

Active / Last Shift

Endpoint: GET /v1/shifts/active?driverId=1

Endpoint: GET /v1/shifts/last?driverId=1

Both return the same shift shape shown above, scoped to the current or most recently completed shift for the given driverId.

5. Orders & Support Chat Module

Method	Endpoint	Purpose
POST	/v1/orders	Create a new delivery order
GET	/v1/orders	Retrieve all orders (filterable by driverId / status)
GET	/v1/orders/:id	Retrieve a single order
PATCH	/v1/orders/:id/status	Update an order's delivery status
POST	/v1/orders/:id/chat	Send a support message on an order
GET	/v1/orders/:id/chat	Retrieve the chat history for an order

Create Delivery Order

Endpoint: POST /v1/orders

Request Body

```
{
  "pickupAddress": "123 Main St",
  "deliveryAddress": "456 Oak Ave",
  "driverId": "1"
}
```

Response

```
{
  "id": "1",
  "pickupAddress": "123 Main St",
  "deliveryAddress": "456 Oak Ave",
  "status": "PENDING",
  "driverId": "1",
  "createdAt": "2026-07-16T10:00:00.000Z"
}
```

Update Order Status

Endpoint: PATCH /v1/orders/:id/status

Request Body

```
{
  "status": "DELIVERED"
}
```

Response

```
{
  "id": "1",
  "status": "DELIVERED",
  "updatedAt": "2026-07-16T10:30:00.000Z"
}
```

```
}
```

Get All Orders

Endpoint: GET /v1/orders

Query Parameters: driverId (optional), status (optional)

Response

```
[
  {
    "id": "1",
    "pickupAddress": "123 Main St",
    "deliveryAddress": "456 Oak Ave",
    "status": "DELIVERED",
    "driverId": "1"
  }
]
```

Support Chat

Each order carries its own support chat thread. Drivers or support staff can post a message with POST /v1/orders/:id/chat, and the full ordered history is available via GET /v1/orders/:id/chat, backed by the ChatMessage entity (orderId, sender, message, createdAt).

6. Wallet Module

Method	Endpoint	Purpose
POST	/v1/wallet/withdraw	Withdraw funds from the driver's wallet
GET	/v1/wallet/transactions	Retrieve transaction history
GET	/v1/wallet/rates	Retrieve cached currency exchange rates

Withdraw Funds

Endpoint: POST /v1/wallet/withdraw

Request Body

```
{
  "driverId": "1",
  "amount": 50.00,
  "currency": "USD"
}
```

Response

```
{
  "id": "1",
```

```
  "driverId": "1",
  "amount": 50.00,
  "currency": "USD",
  "status": "COMPLETED",
  "transactionDate": "2026-07-16T10:00:00.000Z"
}
```

Transaction History

Endpoint: GET /v1/wallet/transactions?driverId=1

Response

```
[
  {
    "id": "1",
    "driverId": "1",
    "amount": 50.00,
    "currency": "USD",
    "status": "COMPLETED",
    "transactionDate": "2026-07-16T10:00:00.000Z"
  }
]
```

Exchange Rates (Cached)

Endpoint: GET /v1/wallet/rates

Rates are fetched from a third-party source and cached via Cache Manager to avoid redundant external calls (Task 3 demonstration).

Response

```
{
  "USD": 1.00,
  "EUR": 0.92,
  "GBP": 0.78,
  "NGN": 1550.00,
  "cachedAt": "2026-07-16T10:00:00.000Z"
}
```

7. Authentication

Current implementation: Work ID-based authentication, supplied via a request header.

```
x-work-id: DRV-123
```

JWT-based authentication and role-based authorization are planned for a future release (see Section 12).

8. Validation

Every write endpoint validates its input before touching the database. The backend enforces:

- Required fields are present
- UUID / ID format correctness

- Numeric fields (amounts, totals) are valid numbers
- Enum values (e.g. order status, transportation type) are within the allowed set
- Work IDs are unique at registration
- Wallet balance is sufficient before a withdrawal is processed
- A shift is active before it can be stopped, and not already active before it is started

9. Data Layer & Caching

Persistent storage is implemented using:

- Prisma ORM
- SQLite
- Better SQLite3 driver

Entities modeled in the schema include:

- User (Drivers)
- Order
- Transaction
- Shift
- ChatMessage

Frequently requested but slow-changing data — the currency exchange-rate list used by the wallet — is cached with NestJS Cache Manager, avoiding redundant calls to the third-party rates provider on every request.

10. API Documentation (Swagger)

All endpoints are documented and testable through an interactive Swagger UI, served at /docs. The screenshots below show the live, generated documentation for the Drivers, Shifts, Orders & Support Chat, and Wallet modules, along with the request/response DTO schemas.

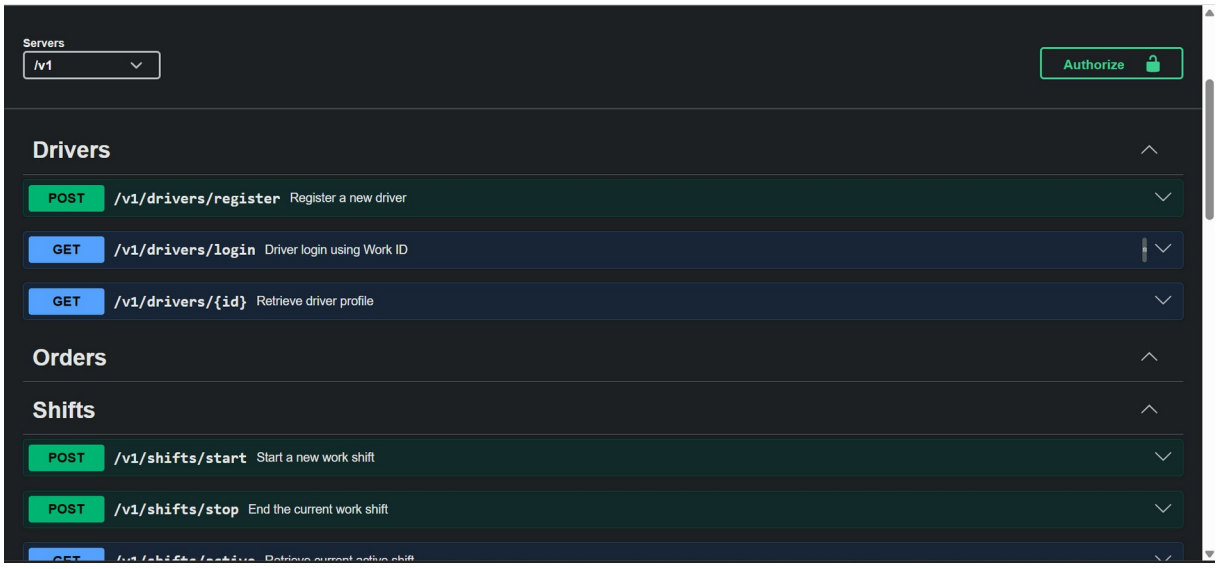

Figure 2 — Swagger UI: Drivers and Shifts endpoints

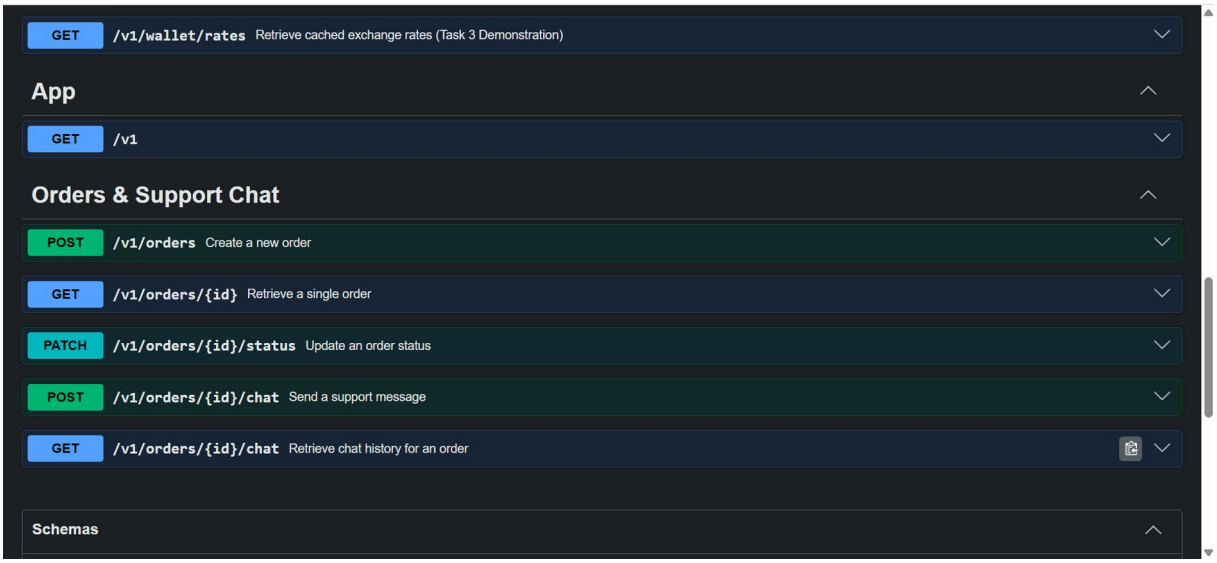

Figure 3 — Swagger UI: Wallet rates, App, and Orders & Support Chat endpoints

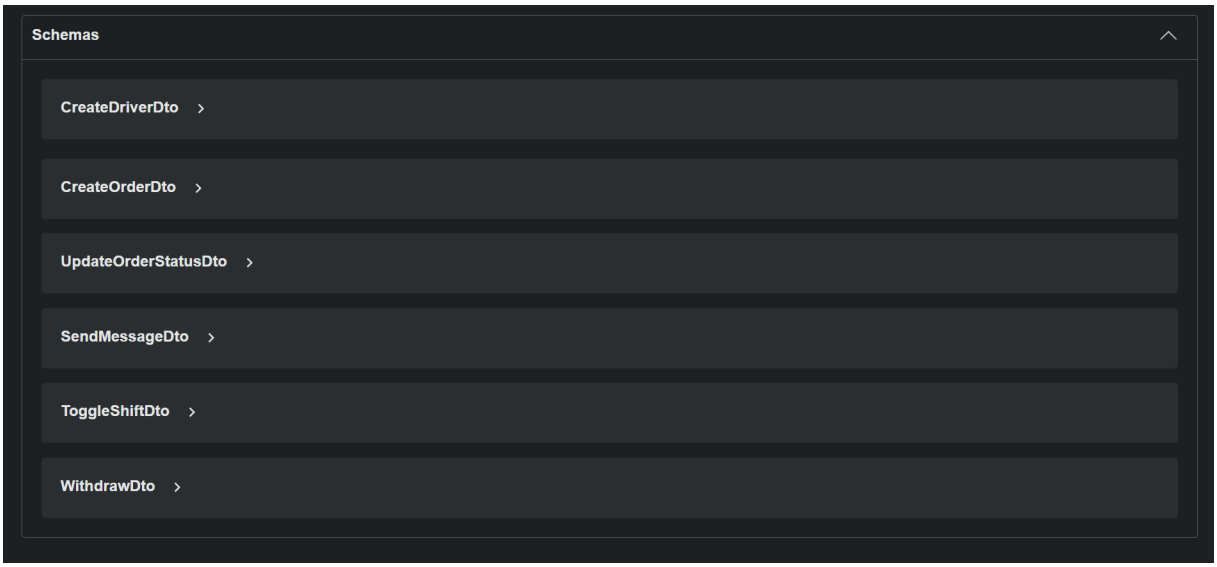

Figure 4 — Swagger schemas (DTOs) used across the API

11. Testing

The backend includes comprehensive end-to-end tests, written with Jest and Supertest, covering the core functionality of every module:

- Driver registration and login
- Driver profile retrieval
- Wallet withdrawals and validation
- Wallet exchange-rate caching
- Shift start / stop / active / last retrieval
- Order creation, retrieval, and status updates
- API boot / startup validation

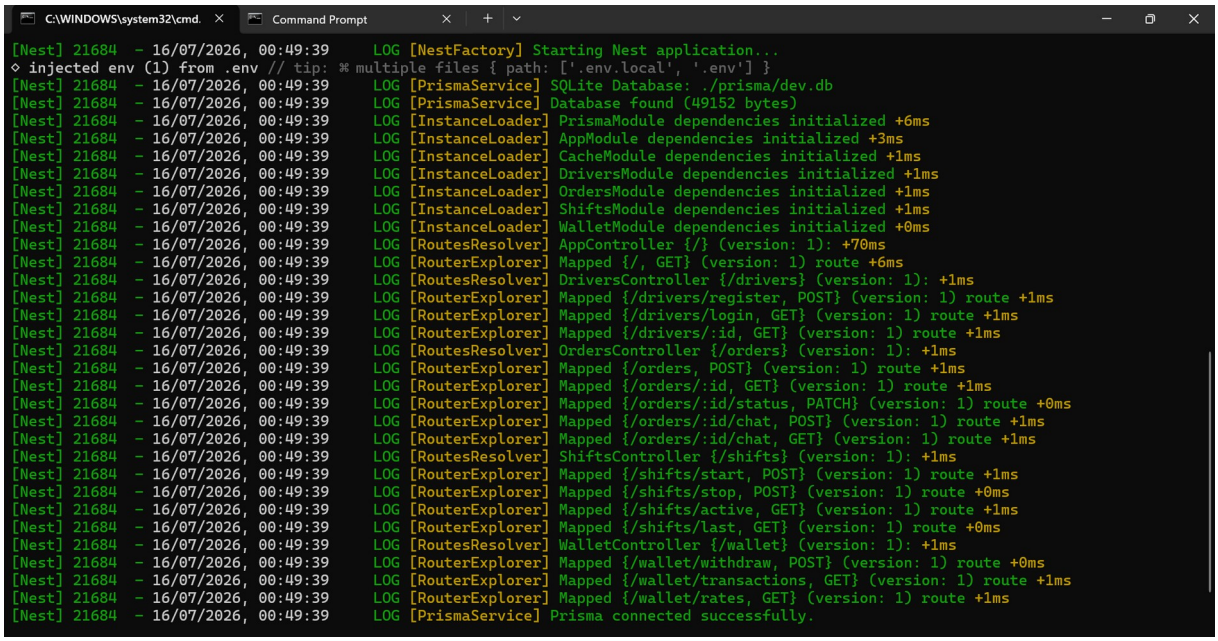

Figure 5 — Application bootstrapping: all modules and routes mapped successfully

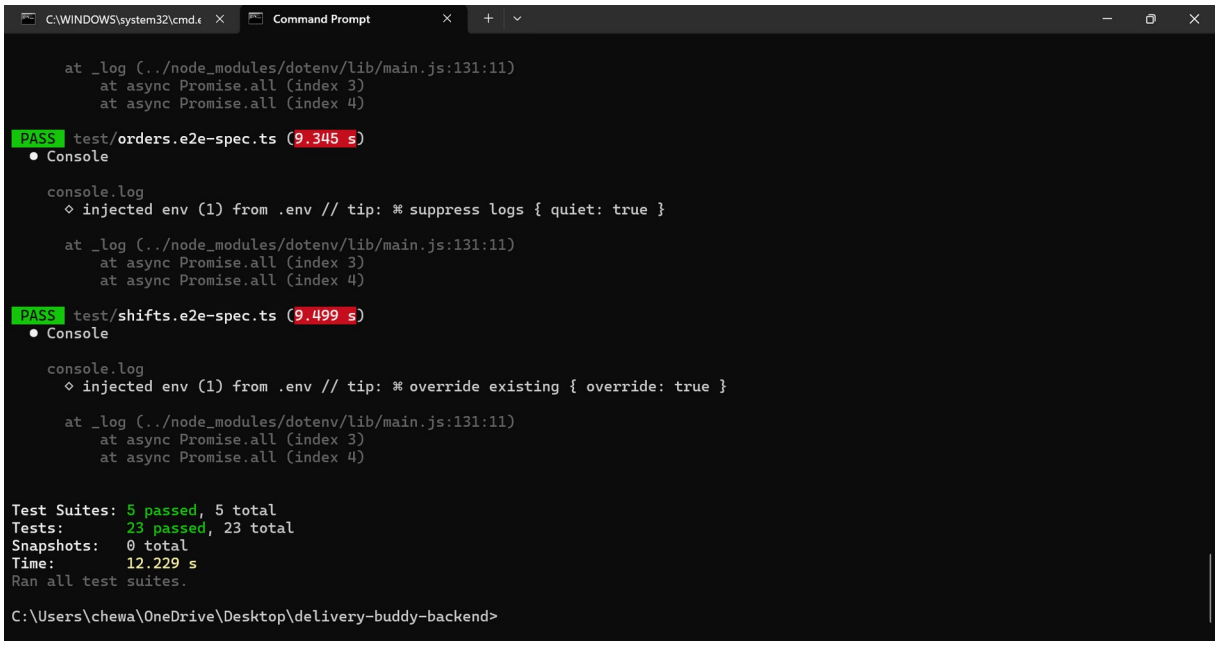

Figure 6 — End-to-end test run: 5 test suites, 23 tests, all passing

Current Results

Metric	Result
Test Suites	5 passed, 5 total
Tests	23 passed, 23 total
Failures	0

12. Technologies Used

Technology	Purpose
NestJS	Backend application framework
Prisma ORM	Database ORM / schema migrations
SQLite	Persistent relational database
Better SQLite3	SQLite driver
Swagger	Interactive API documentation
Jest	Test runner for end-to-end tests
Supertest	HTTP assertions for e2e tests
Cache Manager	Response / exchange-rate caching

13. Setup & Run Instructions

Installation

```
git clone https://github.com/Larry-Craig/delivery-buddy-backend.git
cd delivery-buddy-backend
npm install
npx prisma generate
npx prisma migrate dev
npm run start:dev
```

Environment Variables

```
DATABASE_URL="file:./dev.db"
PORT=3000
NODE_ENV=development
```

Running Tests

```
npm run test:e2e
npm run test:e2e -- --testPathPattern=drivers
```

Troubleshooting

Issue	Solution
Prisma Client not found	Run npx prisma generate
Migration fails	Delete prisma/dev.db and re-migrate
Port already in use	Change PORT in .env file
Database connection error	Ensure DATABASE_URL is set correctly

14. Future Improvements

- JWT authentication
- Role-based authorization
- PostgreSQL deployment for production
- Docker support
- CI/CD pipeline
- Redis caching
- Real-time driver tracking
- Push notifications

15. Author & Support

Author: Chewachong Larry Che (Larry-Craig)

GitHub: https://github.com/Larry-Craig

Support: chewachongcraig@gmail.com